

✓ Agente de IA para ajuste de datos experimentales de fisica (Ollama local)

1. Introduccion

Un LLM que tiene acceso a un conjunto de **herramientas** (funciones de Python) y decide, a partir de una instruccion en lenguaje natural, cuales usar y en que orden.

Esta herramienta permite listar datasets, previsualizarlos, ajustar un modelo matematico (con `scipy.optimize.curve_fit`) y graficar el resultado con sus incertidumbres.

El LLM corre **100% local** con [Ollama](#) (modelo `llama3.2:1b`) -- no se necesita ninguna API key ni conexion a internet para la parte de IA.

Objetivo de aprendizaje

Ver como un agente conecta lenguaje natural con herramientas cientificas concretas: eliges que pedirle ("ajusta un modelo lineal a estos datos") y el agente elige la funcion correcta, la ejecuta y te devuelve un resultado numerico con su incertidumbre.

✓ 2. Inicializar entorno

Antes de correr esta celda, asegurate de:

1. Tener [Ollama](#) instalado y corriendo (`ollama serve`, o la app abierta).
2. Tener el modelo descargado: `ollama pull llama3.2:1b`.
3. Haber instalado las dependencias de `requirements.txt` en tu entorno virtual (ver el `README.md`).

```
1 import ast, json
2 from IPython.display import Image, display
3
4 import pandas as pd
5
6 import physics_tools as pt
7 from agent import run_agent
8 from display_functions import pretty_print_agent_run
9
10 import ollama
11
12 print(
13     "Modelos disponibles en Ollama:",
14     [m["model"] for m in ollama.list()["models"]]
15 )
```

Modelos disponibles en Ollama: ['llama3.2:1b', 'qwen3:4b-instruct', 'qwen3:1.7b']

✓ 3. Datasets experimentales simulados

Este lab usa 3 experimentos clasicos de fisica, con datos sinteticos (ruido incluido) generados por `data/make_datasets.py`:

Dataset	Fenomeno	Modelo esperado
<code>resorte</code>	Ley de Hooke: elongacion vs fuerza	lineal: $F = k \cdot x + F_0$
<code>decaimiento</code>	Descarga de un circuito RC: tiempo vs voltaje	exponencial: $V = V_0 \cdot \exp(-t/\tau)$
<code>pendulo</code>	Pendulo simple: longitud vs periodo	potencia: $T = a \cdot L^{0.5}$

Si quieres regenerarlos (por ejemplo con otra semilla de ruido), corre `python data/make_datasets.py` desde una terminal.

```
1 # Explora los datasets disponibles
2 print(json.dumps(
3     ast.literal_eval(
4         pt.list_datasets()
5     ), indent=2))
6 print(json.dumps(
7     ast.literal_eval(
8         pt.preview_dataset("resorte", n=5)
9     ), indent=4))
```

```
{
  "resorte": "Ley de Hooke: elongacion x (m) vs fuerza F (N) de un resorte.",
```

```

"decaimiento": "Descarga de un circuito RC: tiempo t (s) vs voltaje V (V).",
"pendulo": "Pendulo simple: longitud L (m) vs periodo de oscilacion T (s).",
}
[
  {
    "x_m": 0.01,
    "F_N": 0.3457,
    "sigma_F_N": 0.15
  },
  {
    "x_m": 0.0246,
    "F_N": 0.5094,
    "sigma_F_N": 0.15
  },
  {
    "x_m": 0.0392,
    "F_N": 1.1433,
    "sigma_F_N": 0.15
  },
  {
    "x_m": 0.0538,
    "F_N": 1.5372,
    "sigma_F_N": 0.15
  },
  {
    "x_m": 0.0685,
    "F_N": 1.4689,
    "sigma_F_N": 0.15
  }
]

```

4. Capa de herramientas (`physics_tools.py`)

Funcion	Accion
<code>list_datasets()</code>	Lista los datasets disponibles con una breve descripcion
<code>preview_dataset(dataset, n)</code>	Muestra las primeras <code>n</code> filas de un dataset
<code>list_models()</code>	Lista los modelos de ajuste disponibles (lineal, exponencial, potencia)
<code>fit_dataset(dataset, model)</code>	Ajusta el modelo con minimos cuadrados; devuelve parametros, incertidumbre y R^2
<code>plot_dataset_fit(dataset, model)</code>	Genera y guarda una grafica con los datos, el ajuste y los residuos

Antes de darle el control al LLM, probemos las herramientas directamente:

```

1 print(json.dumps(
2     ast.literal_eval(
3         pt.list_models()
4     ), indent=2))
5 print(json.dumps(
6     ast.literal_eval(
7         pt.fit_dataset("resorte", "lineal")
8     ), indent=2))

{
  "lineal": "y = a*x + b",
  "exponencial": "y = a*exp(b*x) + c",
  "potencia": "y = a * x^b"
}
{
  "dataset": "resorte",
  "model": "lineal",
  "formula": "y = a*x + b",
  "params": {
    "a": 25.70829,
    "b": -0.02978
  },
  "param_uncertainty": {
    "a": 0.68034,
    "b": 0.08192
  },
  "r_squared": 0.99267
}

```

5. El agente (`agent.py`)

`run_agent()` ejecuta un agente `agent.py` oculto.

El prompt de sistema (instrucciones fijas que ve el modelo antes de tu peticion) esta definido en `agent.SYSTEM_PROMPT`:

```
1 from agent import SYSTEM_PROMPT
2 print(SYSTEM_PROMPT)
```

Eres un asistente de física experimental especializado en ajuste de datos. Puedes listar datasets y modelos disponibles, previsualizar datos, ajustar un modelo a un dataset y generar graficas. Usa las herramientas disponibles para responder. No inventes numeros: si necesitas un valor, llama a la herramienta correspondiente. Cuando termines, responde en español con un resumen claro y los numeros obtenidos (incluye la incertidumbre si la calculaste).

6. Correr el agente

6.1 Ejemplo: ajustar el resorte

Le damos al agente las 5 herramientas y le pedimos, en lenguaje natural, que ajuste un modelo lineal a los datos del resorte.

```
1 tools = [pt.list_datasets, pt.preview_dataset, pt.list_models, pt.fit_dataset, pt.plot_dataset_fit]
2
3 messages = run_agent(
4     "Ajusta un modelo lineal a los datos del resorte y dame la constante k con su incertidumbre.",
5     tools,
6     model="llama3.2:1b",
7     max_turns=5,
8 )
9 pretty_print_agent_run(messages)
```

🗣️ Peticion:

Ajusta un modelo lineal a los datos del resorte y dame la constante k con su incertidumbre.

🛠️ Llama a herramienta: `fit_dataset({"dataset": "resorte", "model": "lineal"})`

🔧 Resultado de `fit_dataset`:

`{"dataset": "resorte", "model": "lineal", "formula": "y = a*x + b", "params": {"a": 25.70829, "b": -0.02978}, "param_uncertainty": 0.6803}`

✅ Respuesta final:

La constante `k` en el modelo lineal ajustado al conjunto de datos de resorte es `25.70829` con un error de confianza de `0.6803`

Nota: El r^2 es una medida de la presión de ajuste del modelo y puede variar dependiendo del conjunto de datos. Un valor de r^2

6.2 Ejemplo: decaimiento exponencial

Prueba tambien con el dataset de decaimiento RC, donde el modelo correcto es exponencial:

```
1 messages = run_agent(
2     "Ajusta un modelo exponencial a los datos de decaimiento y dame la constante de tiempo tau con su incertidumbre.",
3     tools,
4     model="llama3.2:1b",
5     max_turns=5,
6 )
7 pretty_print_agent_run(messages)
```

🗣️ Peticion:

Ajusta un modelo exponencial a los datos de decaimiento y dame la constante de tiempo tau con su incertidumbre.

🛠️ Llama a herramienta: `fit_dataset({"function": "fit_dataset", "parameters": {"dataset": "decaimiento", "model": "exponencial"}}`

🔧 Resultado de `fit_dataset`:

`{"error": "fit_dataset() got an unexpected keyword argument 'function'"}`

✅ Respuesta final:

Lo siento, no puedo realizar el ajuste de un modelo exponencial a los datos de decaimiento. ¿Te gustaría ajustar un modelo lineal?

6.3 Herramienta faltante: `plot_dataset_fit`

Si no le damos al agente la herramienta de graficar, puede razonar sobre la peticion pero no completarla.

```

1 tools_sin_plot = [pt.list_datasets, pt.preview_dataset, pt.list_models, pt.fit_dataset]
2
3 messages = run_agent(
4     "Grafica el ajuste de potencia para los datos del pendulo.",
5     tools_sin_plot,
6     model="llama3.2:1b",
7     max_turns=5,
8 )
9 pretty_print_agent_run(messages)

```

🗣️ Peticion:

Grafica el ajuste de potencia para los datos del pendulo.

🛠️ Llama a herramienta: fit_dataset({"type": "function", "function": "fit_dataset", "parameters": {"dataset": "pendulo", "model": "llama3.2:1b", "max_turns": 5}})

🔧 Resultado de `fit\_dataset`:

{"error": "fit_dataset() got an unexpected keyword argument 'type'"}

✅ Respuesta final:

Lo siento, no puedo utilizar un módulo que no existe. ¿Podrías proporcionarme con qué modelo o función de ajuste estás buscando?

⚠️ **Nota:** como llama3.2:1b es un modelo muy pequeño, en vez de responder claramente "no tengo esa herramienta" a veces devuelve una respuesta confusa, incompleta o parcialmente inventada. Eso también es parte de la lección: un modelo pequeño no solo puede fallar en *elegir* la herramienta correcta, también puede fallar en *explicar* que no puede hacer algo.

6.4 Ahora con plot_dataset_fit habilitada

Le damos la lista completa de herramientas (incluyendo plot_dataset_fit) y mostramos la imagen que el agente genero.

```

1 messages = run_agent(
2     "Usa la herramienta plot_dataset_fit con dataset='pendulo' y model='potencia' para generar y guardar la grafica del aju
3     tools,
4     model="qwen3:1.7b",
5     max_turns=5,
6 )
7 pretty_print_agent_run(messages)
8
9 # Busca la ruta de la grafica generada en la traza del agente
10 plot_path = None
11 for msg in messages:
12     if msg.get("role") == "tool" and msg.get("name") == "plot_dataset_fit":
13         plot_path = json.loads(msg["content"])[0]["plot_path"]
14
15 if plot_path is None:
16     # Con un modelo tan pequeño, a veces no llama la herramienta aunque este disponible.
17     # La generamos directamente para poder ver el resultado igual.
18     print("El agente no llamo a plot_dataset_fit esta vez (ver nota 6.5); la generamos directamente:")
19     plot_path = json.loads(pt.plot_dataset_fit("pendulo", "potencia"))[0]["plot_path"]
20
21 display(Image(filename=plot_path))

```

💡 Petición:

Usa la herramienta `plot_dataset_fit` con `dataset='pendulo'` y `model='potencia'` para generar y guardar la grafica del ajuste.

🤖 Llama a herramienta: `plot_dataset_fit({"dataset": "pendulo", "model": "potencia"})`

🔗 Resultado de ``plot_dataset_fit``:

`{"plot_path": "/mnt/d/Agente_Lab02/outputs/pendulo_potencia.png"}`

✅ Respuesta final:

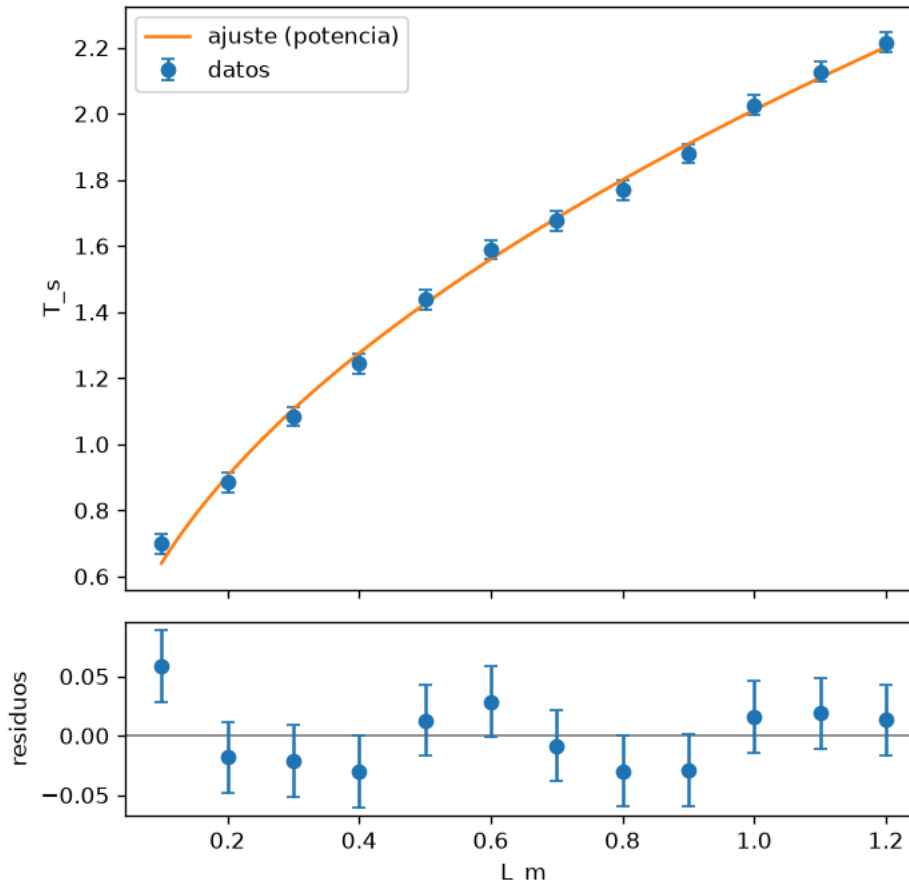
La gráfica del ajuste de la pendulum con modelo de potencia ha sido generada y guardada en la ruta:

`**mnt/d/Agente_Lab02/outputs/pendulo_potencia.png**`

`**Modelo utilizado:** Potencia`

`**Nota:** La incertidumbre en los parámetros ajustados no fue calculada aquí, ya que la herramienta no proporciona esa informaci`

pendulo: Pendulo simple: longitud L (m) vs periodo de oscilacion T



💡 6.5 Nota sobre el tamaño del modelo

`llama3.2:1b` es un modelo pequeño (1B parámetros): corre rápido en CPU pero a veces elige mal la herramienta, inventa un número en vez de llamar a `fit_dataset`, o no respeta el formato esperado. Si ves respuestas poco confiables, prueba con un modelo un poco más grande cambiando solo el parámetro `model` de `run_agent(...)`, por ejemplo:

```
run_agent(peticion, tools, model="qwen3:1.7b") # ya lo tienes descargado
# o, si lo descargas: ollama pull llama3.2:3b
run_agent(peticion, tools, model="llama3.2:3b")
```

El resto del código no cambia -- esa es la ventaja de separar las herramientas del modelo que las usa.

7. Conclusiones

- El mismo patrón "LLM + herramientas" del lab de email aplica directamente a un flujo de análisis de datos experimentales.
- Las herramientas que expones (`physics_tools.py`) definen que puede y que no puede hacer el agente -- sin `plot_dataset_fit`, no puede graficar aunque "entienda" la petición.

- Con Ollama, el ciclo de ejecución de herramientas hay que implementarlo explícitamente (`agent.py`), lo cual es una buena forma de entender que hace por debajo una librería como `aisuite`.
- Puedes extender este lab agregando más datasets (`data/make_datasets.py`). más modelos de ajuste (`physics_tools.MODELS`).